

Lecture 07: Strings

Understand how strings are organized, working with strings, and explore various string methods

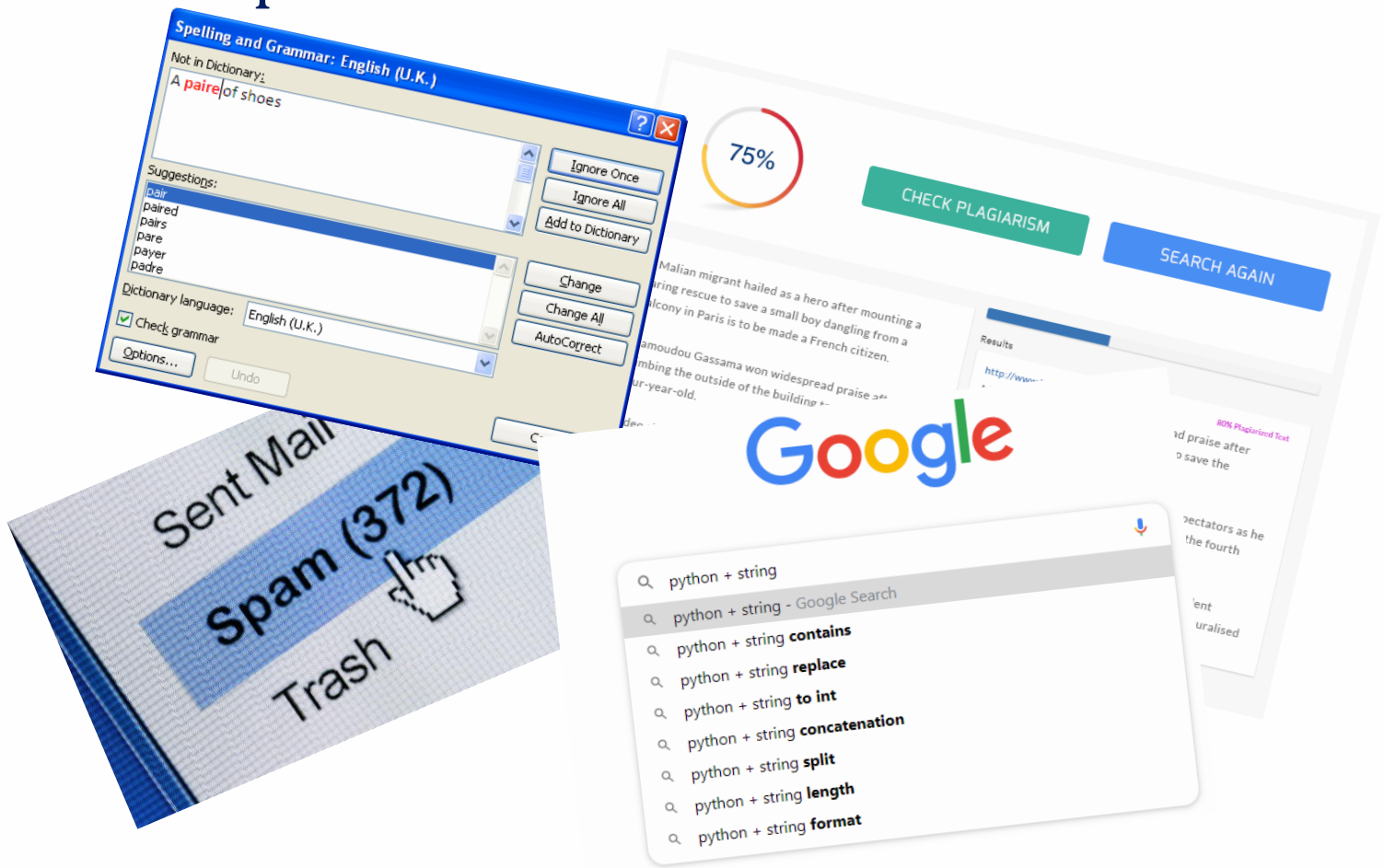
Pham Huy Hieu & Kok-Seng Wong | Fall 2025

Tuesday 7, 2025

MOTIVATIONAL EXAMPLES 2

"String matching" for real world problems:

- Spam Filters
- Search Engines
- Spell Checkers
- Plagiarism Checkers
- Information Retrieval
- etc.



→ We need to know "strings manipulation"!

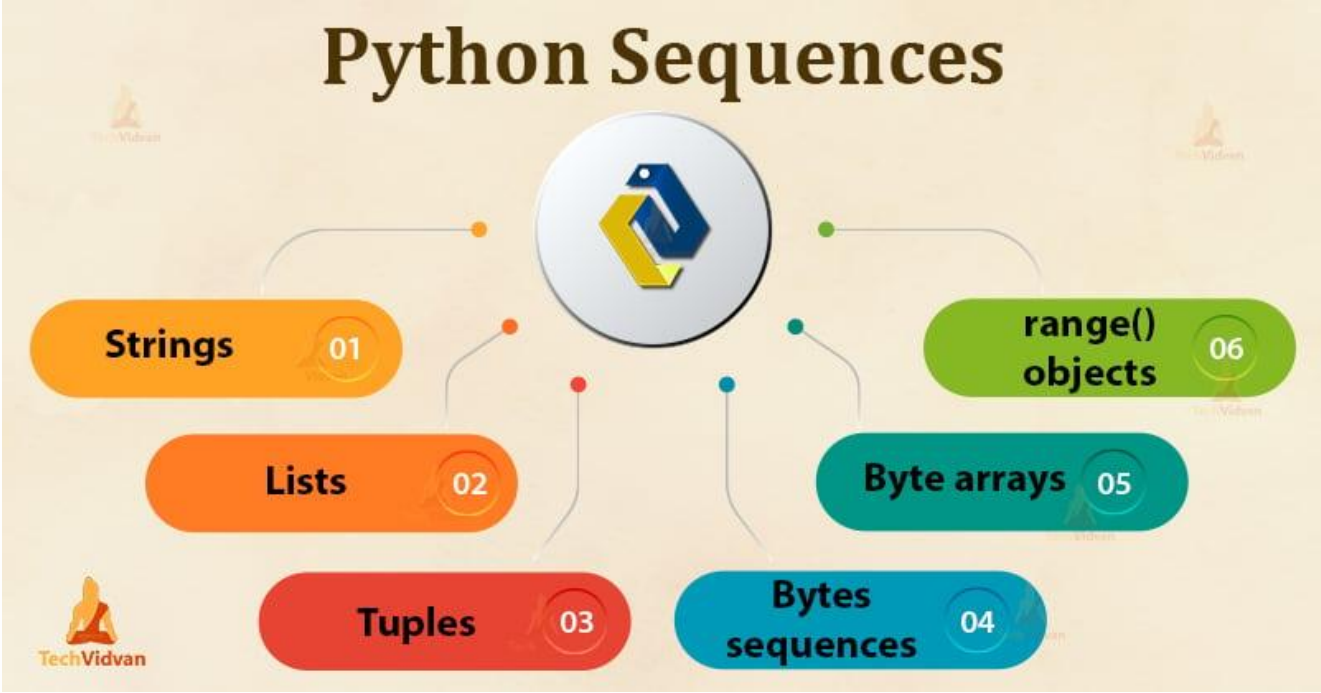
LEARNING OUTCOMES

Upon the completion of this lecture, students will be able to:

- explain the important of strings manipulation in real world applications.
- understand and apply string slicing methods to create a substring from the source string and original string.
- use different operators and built-in methods that work with strings.

SEQUENCE

- In Python, a sequence is an ordered collection of items, where each item can be accessed by its position (index) within the sequence.
- There are six types of sequence:
  - Strings
  - Unicode strings
  - Lists
  - Tuples
  - xrange objects



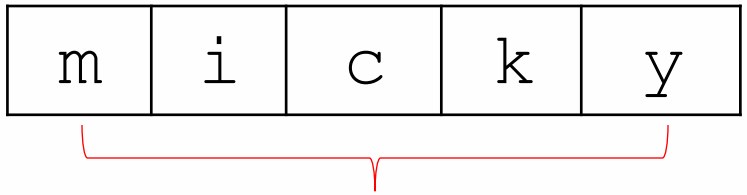
Source

STRINGS

- A sequence of case sensitive characters.
- Comparison operators: `==`, `!=`, `>=`, `<=`, `>`, `<` etc.
- `len()`: a function used to retrieve the length (or size) of the string and returns to an `int`

```
>>> s = "micky"
>>> len(s)
5
>>> len("Ali Baba")
?
```

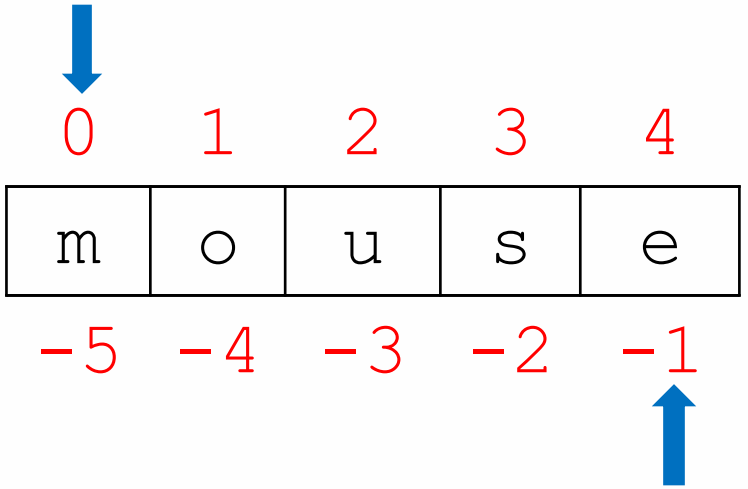
Common mistake: `lens()`



STRINGS ARE INDEXED

- `x = "mouse"`

index: always starts at 0



index: last element
always at index -1

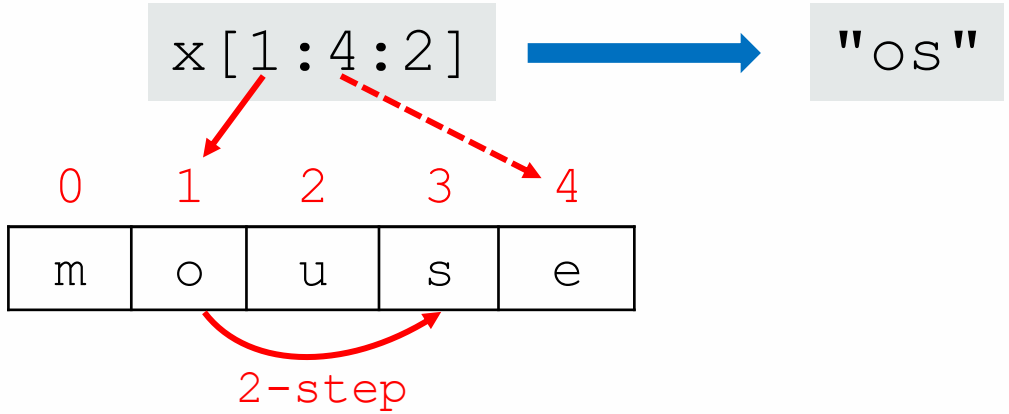
x[0]	m
x[1]	o
x[2]	u
x[3]	s
x[4]	e
x[-5]	
x[-4]	
x[-3]	
x[5]	?

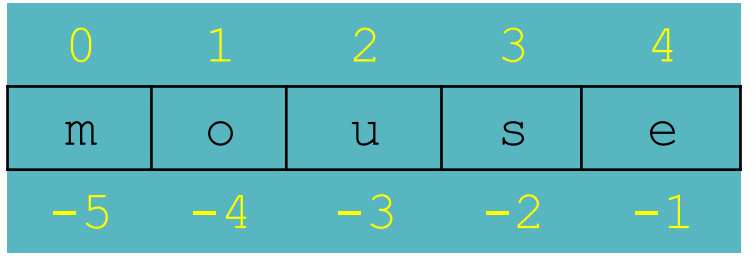
```
>>> x[5]
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
IndexError: string index out of range
```

- We can access each character in `x` using `[]`

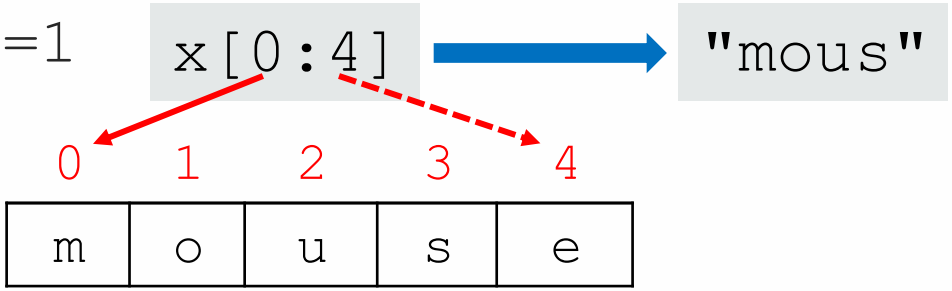
STRING SLICING

- Slice string with `[start:stop:step]`





- If `[start:stop]`, by default `step=1`



- Can omit numbers and use just colons:

`x[:]` → Same as `x[0:len(x)]`

`x[::]` → Same as `x[0:len(x):1]`

CLASS PRACTICE

Sample Code:

```
x[:]
x[::]
x[::-1]
x[0:4]
x[4:1:-2]
x[1:4:2]
x[3::-2]
```

Output:

0	1	2	3	4
m	o	u	s	e
-5	-4	-3	-2	-1



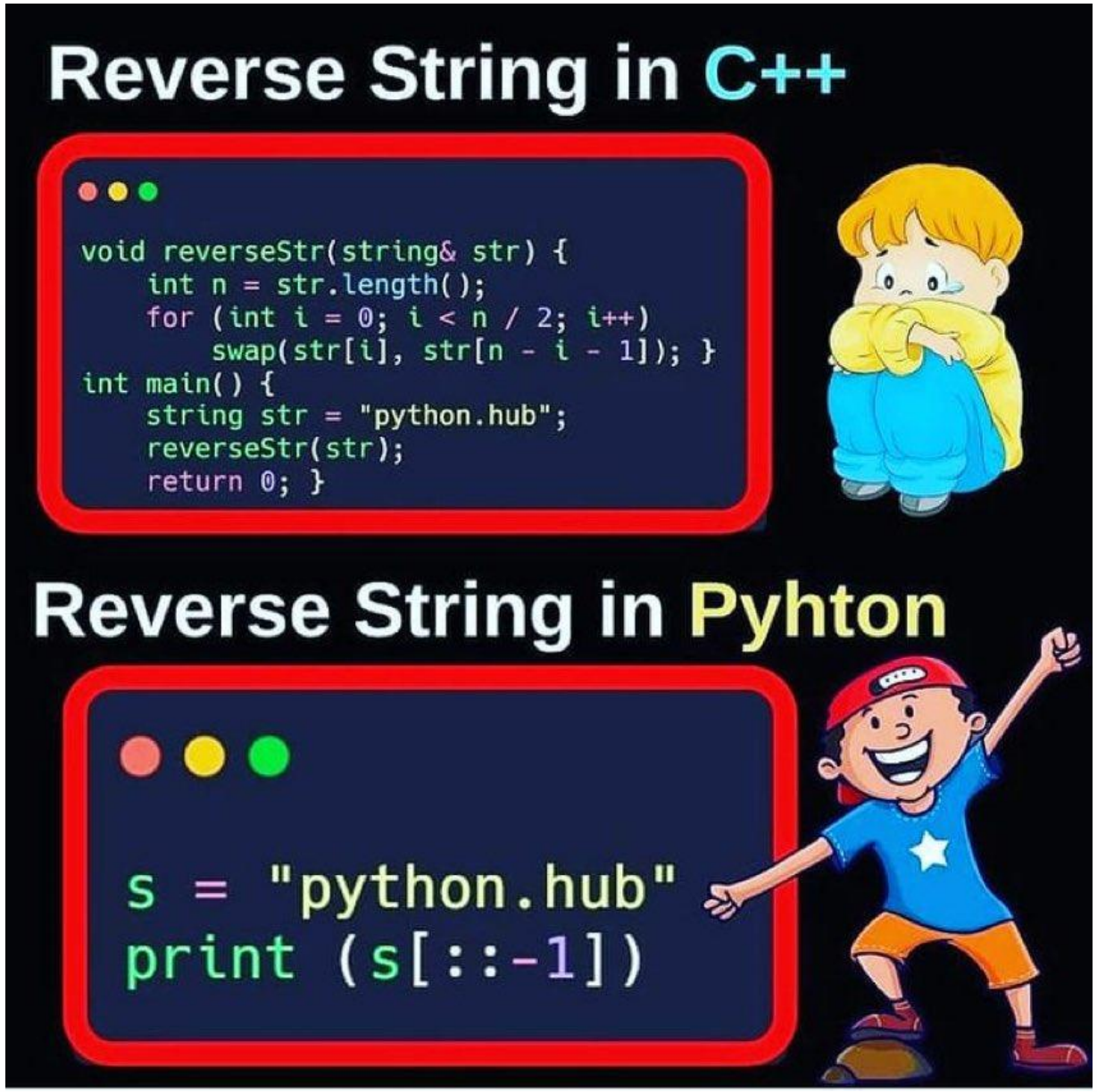
CLASS PRACTICE

Sample Code:

```
len(x[:])
len(x[::])
len(x[::-1])
len(x[0:4])
len(x[4:1:-2])
len(x[1:4:2])
len(x[3::-2])
```

Output:

0	1	2	3	4
m	o	u	s	e
-5	-4	-3	-2	-1



[source]

```
>>> x="Ali Baba"
>>> x[:]

>>> x[::]

>>> x[::len(x)]

>>> x[::-1]
```

STRINGS ARE IMMUTABLE

- Cannot modified the elements in a string:

strings and tuples are immutable but, lists and dictionaries are mutable.

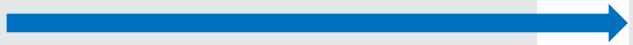
```
y = "Hello"
y[0] = "M"
```



```
TypeError: 'str' object does not support item assignment
```

- But strings can bound to a new object:

```
y = "M" + y[1:len(y)]
print(y)
```



```
"Mello"
```

CLASS PRACTICE

Sample Code:

```
>>> a = "Micky Mouse"
>>> b = "Donald Duck"
>>> c = "Bruce Lee"
>>> a = a[0] + b[1:4] + c[5:7] + "isa"
>>> print(a)
>>> a[1]
```

Output:

CLASS PRACTICE

Sample Code:

```
>>> a = "Micky Mouse"
>>> b = "Donald Duck"
>>> c = "Bruce Lee"
>>> b = b[-6::-1] + a[-9::-2]
>>> c = c[6:10] + a[5] + "loves " + b[::-1]
>>> print(c)
```

Output:

STRING "IN" OPERATOR

- Operator: `in`
- To test if `str1` is a substring of `str2`
- Evaluates to a **`True or False`**

```
>>> str1 = "I'm selling apple, orange, durian, banana!"
>>> str2 = "banana"
>>> str2 in str1
True
>>> "Orange" in str1
False
>>> "m" in str2
False
>>> "" in str1
True
```

Python has `not in` operator too!

STRING "IS" OPERATOR

- Operator: `is`
- To check if both `str1` and `str2` refer to the same object
- Evaluates to a **`bool`**

```
>>> str1 = "durian"
>>> str2 = str1[1:2]
>>> str2 is str1
False
>>> str3 = str1[:]
>>> str3 is str1
True
```

```
>>> id(str1)
1882887600560
>>> id(str3)
1882887600560
```

CLASS PRACTICE

Sample Code:

```
>>> str1 = "racecar"
>>> str2 = str1[::-1]
>>> str1 in str2
```

Output:

CLASS PRACTICE

Write a function called `my_string` to get a string made of

(a) 3 copies of the first 2 characters of a given string.

```
>>> my_string(”VinUniversity”)
ViViVi

>>> my_string(”Cornell”)
CoCoCo
```

(b) 3 copies of the last 3 characters of a given string.

```
>>> my_string(”VinUniversity”)
ityityity

>>> my_string(”Cornell”)
ellellell
```

STRING METHODS

- Python has a set of built-in methods that we can use on strings.
- All string methods returns new values. They do not change the original string.
- Format: `<string name>.<method name>(x,y,…)`
- Some popular string methods:

```
upper(), index(), count(), isupper(), strip(), etc.
```

Complete list: https://docs.python.org/2.4/lib/string-methods.html

PYTHON BUILT-IN FUNCTIONS FOR STRINGS

Function	Description	Example
`len()`	Returns the length of the string	`len("Hello")` → `5`
`str()`	Converts an object to a string	`str(123)` → `"123"`
`format()`	Formats a string using placeholders	`"Name: {}, Age: {}".format("Alice", 30)` → `"Name: Alice, Age: 30"`
`ord()`	Returns Unicode code point of a character	`ord('A')` → `65`
`chr()`	Returns character for a given Unicode code	`chr(65)` → `'A'`
`lower()`	Converts all characters to lowercase	`"HELLO".lower()` → `"hello"`
`upper()`	Converts all characters to uppercase	`"hello".upper()` → `"HELLO"`
`capitalize()`	Capitalizes the first character	`"hello world".capitalize()` → `"Hello world"`



PYTHON BUILT-IN FUNCTIONS FOR STRINGS

Function	Description	Example
`title()`	Capitalizes the first character of each word	`"hello world".title()` → `"Hello World"`
`find()`	Returns index of first occurrence of substring, or `-1` if not found	`"Hello".find("e")` → `1`
`index()`	Returns index of first occurrence, raises `ValueError` if not found	`"Hello".index("l")` → `2`
`replace()`	Replaces occurrences of a substring	`"Hello".replace("H", "J")` → `"Jello"`
`split()`	Splits string into list using delimiter	`"a,b,c".split(",")` → `['a', 'b', 'c']`
`join()`	Joins elements of a list into a string	`",".join(['a', 'b', 'c'])` → `"a,b,c"`
`strip()`	Removes leading and trailing whitespace	`" Hello ".strip()` → `"Hello"`
`lstrip()`	Removes leading whitespace	`" Hello".lstrip()` → `"Hello"`
`rstrip()`	Removes trailing whitespace	`"Hello ".rstrip()` → `"Hello"`



PYTHON BUILT-IN FUNCTIONS FOR STRINGS

Function	Description	Example
`startswith()`	Checks if string starts with the specified prefix	`"Hello".startswith("He")` → `True`
`endswith()`	Checks if string ends with the specified suffix	`"Hello".endswith("lo")` → `True`
`count()`	Returns the number of occurrences of a substring	`"Hello".count("l")` → `2`
`isdigit()`	Checks if all characters are digits	`"12345".isdigit()` → `True`
`isalpha()`	Checks if all characters are alphabetic	`"Hello".isalpha()` → `True`
`isalnum()`	Checks if all characters are alphanumeric	`"Hello123".isalnum()` → `True`
`isspace()`	Checks if all characters are whitespace	`" ".isspace()` → `True`
`zfill()`	Pads the string on the left with zeros	`"42".zfill(5)` → `"00042"`



STRING METHODS EXAMPLE 1

- String method:

`isupper()` → returns True if all characters in the string are upper case

`upper()` → converts a string into upper case

```
>>> s1 = "HELLO"
>>> s1.isupper()
True
>>> s2 = "world"
>>> s2.upper()
'WORLD'
>>> s1 + " " + s2
'HELLO world'
```

STRING METHODS EXAMPLE 2

- String method:

`index()` → searches the string for a specified value and returns the position of where it was found (first occurrence)

- s3 = "Ali Baba"




```
>>> s3.index("A")
>>> s3.index(" ")
>>> s3.index("a")
>>> s3.index("aba")
>>> s3.index("z") ?
>>> s3.index("i",0, 2) ?
```

`str.index()` raises a `ValueError` if the substring you are searching for is not found in the string.

```
text = "Hello, Hello, world!"

# Find the first occurrence of the letter 'o'
index_o = text.index('o')
print(f"The first occurrence of 'o' is at index: {index_o}")

# Find the first occurrence of the substring 'Hello'
index_hello = text.index('Hello')
print(f"The first occurrence of 'Hello' is at index: {index_hello}")

# Find the first occurrence of the letter 'l'
index_l = text.index('l')
print(f"The first occurrence of 'l' is at index: {index_l}")
```

```
The first occurrence of 'o' is at index: 4
The first occurrence of 'Hello' is at index: 0
The first occurrence of 'l' is at index: 2
```

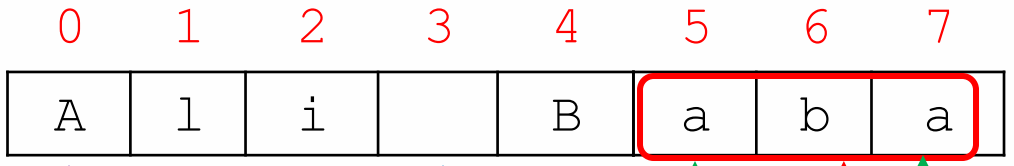


STRING METHODS EXAMPLE 3

- String method:

`find()` → returns the lowest index of the substring if found, and -1 if the substring is not found

- s3 = "Ali Baba"




```
>>> s3.find("A")
>>> s3.find(" ")
>>> s3.find("a")
>>> s3.find("aba")
>>> s3.find("z")  ?
```

Feature	`find()`	`index()`
Return Value	Returns the index if found, `-1` if not	Returns the index if found, raises `ValueError` if not
Error Handling	No exception, returns `-1` if not found	Raises `ValueError` if not found
Usage	Safe for cases where substring might not exist	Used when substring is expected to be present
Performance	Similar when substring is present	Similar when substring is present

Which One to Use?

- **Use** `find()` when the substring might not exist, and you don't want to handle exceptions.
- **Use** `index()` when you're sure the substring should exist and want an error raised if it's missing.

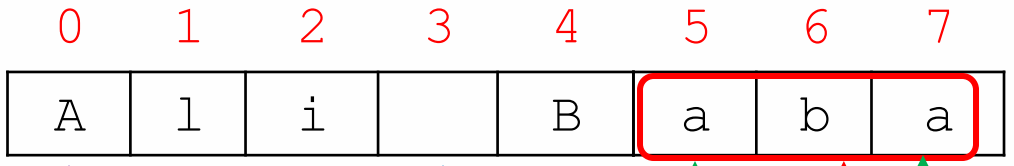


STRING METHODS EXAMPLE 4

- String method:

`count()` → returns the number of times a specified value occurs in a string

- s3 = "Ali Baba"




```
>>> s3.count("A")
>>> s3.count(" ")
>>> s3.count("a")
>>> s3.count("aba")
>>> s3.count("z") ?
```

STRING METHODS EXAMPLE 5

- String method: `strip([chars])`

  ⟶ returns a copy of the string with the leading and trailing characters removed.

- s4 = "Ali Baba "

A	l	i		B	a	b	a	

```
>>> s4.strip()
'Ali Baba'
>>> s4.strip("A")
'li Baba '
>>> s4.strip("A a")
'li Bab'
>>> x.strip("Ali ")
'Baba'
```

If [chars] is not provided, all leading and trailing whitespaces will be removed from the string.

All "A", whitespaces, and "a" will be removed from the left and right of the string.

STRING METHODS EXAMPLE 6

- Split method: `split([separator=None, maxsplit=-1])`

  ⟶ Divides a string into a list of substrings based on a specific delimiter.

- s4 = "Ali Baba "

A	l	i		B	a	b	a	

```
>>> s4.split("")
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ValueError: empty separator
>>> s4.split(" ")
['Ali', 'Baba', '']
>>> s4.split(",")
['Ali Baba ']
```

If the delimiter does not exist, `split()` will return a list containing the original string as a single element.

CLASS PRACTICE

Sample Code:

```
>>> x = "Harry Potter"
>>> y = x.strip("Hter")
>>> y.strip("o yP")
>>> "cat,dog,snack".split(",")
```

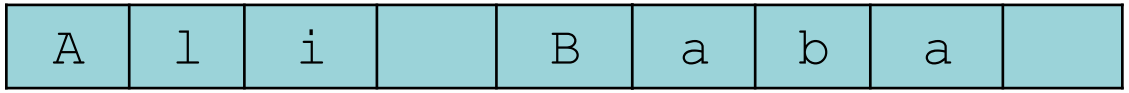
Output:

STRING METHODS EXAMPLE 7

- String method:

`replace()` → returns a string where a specified value is replaced with another value

- s5 = "Ali Baba "




```
>>> s5.replace("Ali", "James")
'James Baba '
```



CLASS PRACTICE

Sample Code:

```
>>> friend = "Ali Baba"
>>> friend.replace("a", "o")
>>> friend.replace("li", "my")
>>> friend.replace("", "-")
>>> friend.replace(" ", "&")
```

Output:

STRING METHODS EXAMPLE 8

- String method:

`isnumeric()` → returns True if all characters in the string are numeric characters

- s6 = "V20205222“

```
>>> s6.isnumeric()
False
>>> s6[1:len(s6)-1].isnumeric()
True
```

- Other methods:

`isalpha(),isalnum(),isdigit(),isdecimal()…`

```
More details:
https://docs.python.org/3.4/library/stdtypes.html?highlight=strip
```

STRING METHODS EXAMPLE 9

`capitalize()` → returns a copy of string with the first character converted to uppercase and all other characters converted to lowercase

```
>>> S7 = "heLLO aLI"
>>> s7.capitalize()
'Hello ali'
```

`title()` → returns a copy of string with the first character of each word converted to uppercase and all other characters converted to lowercase

```
>>> s7.title()
'Hello Ali'
```

STRING: TEXT AS A VALUE

- Strings are written as quoted characters:

```
'Hellow World!' (Python prefers)
"Hellow World!" (most programming languages)
```

- How to write quotes in quotes?

```
'"' or "'"
Both " and '
```

- Use escape charters:

Char	Description
\'	Single quote
\"	Double quote
\n	New line
\t	Tab
\\	backslash

STRING WITH RANDOM()

```
>>> import random
>>> x = "ABCDE"
>>> random.choice(x)
'C'
>>> random.choice(x)
'A'
>>> random.choice(x)
'B'
```

Read More: https://docs.python.org/3/library/random.html